AI-Assisted On-Call Queue: Budgeted Incident Detection for Cloud Microservices

Aishah Mohamed Alyammahi

Abstract

An on-call engineer can read only a few alerts a day, yet most real faults never write a log line, so rules that rank by log severity miss them. This project builds a queue that shows at most K items a day, in the order a live system learns about them, and checks it on the public GAIA microservice benchmark. The queue combines rule-based metric detectors (memory jumps, slow leaks, monitoring gaps) with an optional cost-sensitive logistic-regression classifier for log alerts, and ships as a FastAPI service in one Docker container, in shadow mode by default. On 1,223 test faults at $K = 20$, the detector catches 72.6% (best possible 84.9%) against 68.9% for a Prometheus rate rule and 0% for log severity, and wins all 8 walk-forward weeks with 1.0 instead of 2.1 false items a day. The classifier adds no catches on GAIA, and when the promotion gate is decided on cases that are never scored it refuses the classifier on all six RCAEval applications; plain severity alerts give most of the log-alert gain there. The advantage over the rate rule is modest, and every GAIA fault is the same injected memory fault.

1 Motivation and objectives

Problem. A platform of many small services raises thousands of alerts a day. The engineer on call can read about $K = 20$. Teams escalate by log severity and delete exact duplicates, or use fixed metric thresholds. On GAIA, 82% of the injected faults write no log line and appear only as a jump in container memory, so log severity catches none of them at $K = 20$, and a fixed threshold either misses faults or floods the queue. GAIA injects about 22 faults a day, more than a 20-item budget on 34 of 55 days.

Users and value. The primary users are on-call engineers; secondary users are the operator who decides when the tool may page, and a team that wants to check the tool on its own data before trusting it. The value is a short, ordered, explained list that contains more real faults and fewer false items.

Objectives. O1: beat the rules a team already runs (log severity, static threshold, Prometheus rate rule) at $K = 20$. O2: decide in real time, using no later data. O3: engineer it for production (authentication, safe paging, versioned models with rollback, monitoring, Docker). O4: test it (at least 90% of tests passing, coverage of at least 85%, p95 latency of at most 1 s at 50 users). O5: let another team evaluate it on its own data without code.

2 Architecture and implementation

The code has two layers. `app/src/` is a library with no HTTP (detectors, causal queue, classifier, evaluation); `app/product/` is the web app, job pipeline, connectors, model registry and monitoring. The queue is first come, first served: a point is hot when it is 0.5 GB above the median

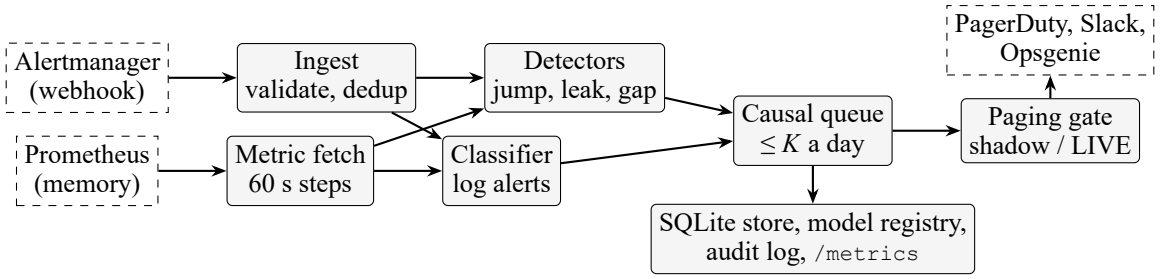


Figure 1: Architecture. Solid boxes run in one container; dashed boxes are external. Engineers' Real/Noise marks flow back through the store into the next model.

of the previous 15 minutes, two hot points within two minutes confirm a jump, and a jump is shown at confirmation only if its rise passes a threshold learned on week 1 (0.62 GB). A 3-hour median catches slow leaks, a service silent for 10 minutes becomes a gap item, and log alerts may use at most a quarter of the slots. Nothing uses a later point, the end of a jump or its peak; `tests/test_causality.py` checks this. Table 1 lists the components and their interfaces.

Component	Responsibility and interface	Main tests
Ingest (`inputs.py`)	Validate and de-duplicate alerts and webhooks; returns `Alert` list	`alertmanager`, `ai_inputs`
Detectors (`memory_spikes.py`)	`detect_jumps/leaks/gaps(metrics, config)` returns `Spike` list	`memory_spikes`
Causal queue (`queue.py`)	`build_queue(items, config)` keeps at most K a day, first come first served	`queue`, `causality`
Classifier, gate (`classifier.py`, `promotion.py`)	Alert probabilities; `promotion_decision(report, incumbent, config)` returns promote or keep	`classifier`, `model_registry`
Paging, store (`paging.py`)	Shadow or LIVE decision per job; SQLite queue, marks, audit log	`would_have_paged`, `security`
Evaluation (`run_*.py`)	Scripts write `results/*.csv`; `run_validate.py` checks the report	`run_validate`, `reproduce_sample`

Table 1: Components, interfaces and the test files (`app/tests/test_*.py`) that cover them.

Design decisions. (1) Rules do the detection and learning is limited to a confirmation threshold and the log-alert classifier, because a rule can be read and audited. (2) Time split only: train on the first 7 days, test on the next 55, never shuffled. (3) Shadow mode is the default; live paging needs a password, a channel and the typed word LIVE. (4) A new classifier is promoted only if, on held-out data, the queue with it costs at least one missed fault less than the queue without log alerts and the queue it replaces (a miss costs 10, a false item 1); the operator can roll back. (5) Models are stored as skops archives, never pickle. (6) A single worker with SQLite, one container per team. **Trade-offs.** Rules are auditable but their jump size must be set for each team's services (the robustness replay shows this); the strict gate refuses a gain of one case, so a useful classifier can be turned away; shadow mode is safe but nothing pages until the operator opts in; SQLite is simple but gives no failover. Technologies: Python 3.12, scikit-learn, FastAPI, SQLite, Docker, uv with a hash-pinned lock, GitHub Actions.

Changes from the proposal. The proposal planned an alert classifier as the main result. Measurement showed that on GAIA the metric rules carry the result, so the classifier became an optional component behind the promotion gate. A code review removed five uses of later-in-the-day information (full queue 70.2% to 69.6%), and a Prometheus integration test found that

a 121 s query step meant live jumps never confirmed; the fetch now uses 60 s steps.

3 Testing, evaluation and results

Methods. The suite has 446 tests: unit tests for detectors, queue and policy; API and pipeline integration tests with outbound calls mocked; AI-specific cases (malformed rows, one-class labels, a worse model not promoted); security tests (login lock, SSRF, CSRF); a Docker integration test against real Prometheus and Alertmanager; and a load test. CI runs lint, type checks, tests on two Python versions, a dependency audit and a Docker build. A script, `run_validate.py`, checks every number in the long report against `app/results/`. Evaluation uses the 55 test days of GAIA MicroSS (1,223 faults), an 8-week walk-forward, bootstrap intervals and sign tests.

System, $K = 20$ items a day	Caught	False/day	Precision
Log severity + exact duplicates	0.0%	0.8	0%
Static rule (week-1 99th percentile)	57.2%	3.0	81%
Prometheus rate rule	68.9%	2.1	88%
Z-score, EWMA, isolation forest (untuned)	19.7%, 32.4%, 19.8%	14.2, 11.4, 0.3	24%, 39%, 94%
Detector alone (jump + learned threshold)	**72.6%**	**1.0**	94%
Full queue with classifier (refused on GAIA)	69.6%	1.8	90%
Shipped queue (log alerts by severity)	71.0%	1.4	92%

Table 2: Faults caught of 1,223 (best possible at $K = 20$: 84.9%). The detector's 95% bootstrap interval is 70.1–75.1%.

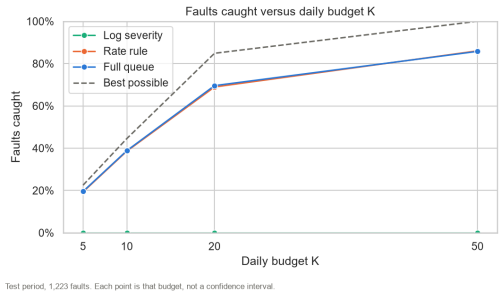


Figure 2: Faults caught versus daily budget K on GAIA. The detector leads at small K and ties the rate rule at 50.

Results. Table 2 shows the detector beats the rate rule in 8 of 8 weeks (sign test $p = 0.0039$) and by 3.7 points overall (interval 2.6 to 4.8), with fewer false items. At $K = 50$ the two are tied (the detector is 0.25 points behind, interval −0.65 to +0.08), so the advantage exists only where slots are scarce. The full queue does not beat it week by week (4 wins, 3 losses, 1 tie). Median time to confirm is 0.72 minutes; only 2% of misses are the detector's fault (45% fall in a monitoring outage, 53% lose to the budget). The classifier ablation is the main negative result: with classifier-chosen log alerts the queue catches 69.6%, with severity-chosen alerts 71.0%, with none 71.2%. On RCAEval, an earlier run promoted the classifier on RE2 Train Ticket, but its gate was run on the cases that were scored. I re-ran it with train, validation and test cases kept apart: the gate then refuses the classifier on all six applications (on Train Ticket it saves 4 against a margin of 10). Pooled over 69 RE2 test cases, the shipped queue, which uses severity alerts, catches 68.1% against 60.9% with no log alerts (+7.2 points, interval +1.5 to +14.5), and the classifier adds one case over severity. A robustness replay shows the learned threshold rejects every 0.5 GB fault, so it should be trained on a team's own incidents. The app serves 50 users with p95 0.32 s and 0% errors.

Beyond GAIA. The same engine, tuned on the first half of each series, was run on NAB, SMD (28 servers) and RCAEval. On SMD it catches 55.1% of 196 incidents against a 12.4% chance line, with low precision (11 false items a day). On NAB it is above chance on traffic and machine data and no better on Twitter volume. On RCAEval RE1 memory faults the budgeted queue catches 0 of 13 on every application. These failures are reported in the repository: the 0.5 GB jump size and the learned threshold must be set for each team's own services.

Against the objectives. Table 3 traces each objective to its design and evidence (test files under `app/tests/`); 10 tests skip (torch, xgboost, CUDA or Docker) and none fail. O1 is met for the detector and tied for the full queue; no live pilot was run.

Objective	Design element	Evidence
O1 beat rules	Jump detector, learned threshold	Table 2; 8 of 8 weeks; `experiment`
O2 real time	Causal queue, no later data	`causality`; median 0.72 min
O3 production	Shadow default, gate, skops, login	`security`, `model_registry`
O4 tested	pytest, CI, load test	436 passed, 87% coverage, p95 0.32 s
O5 other teams	`/evaluate`, NAB, SMD, RCAEval	`generic`, `rcaeval_logs`

Table 3: Objectives traced to design and evidence.

4 Deliverables, setup and use

Repository. `https://github.com/amamsy90/alert-fatigue-reduction`, release 1.1.0 plus the changes in `CHANGELOG.md`, submitted commit: `1a024ebc9c42e3def2d95afe5d340580118f10bc`. The README lists the team, exact commands, the tested system and data sources. No paid API is needed and the system calls no external AI service.

Setup and use. From `app/`: `docker compose up -d` serves the app at `127.0.0.1:8080` in shadow mode; `uv sync --group dev`, `uv run pytest -q` and `uv run python run_validate.py` run the tests and check the reported numbers. `uv run python run_reproduce.py` reproduces the GAIA result on a committed 10-day real sample in under a minute; `--full` uses the 1.8 GB data fetched by `tools/fetch_gaia.py`. Workflow: Alertmanager posts alerts, a job pulls memory from Prometheus and builds the day's queue with a reason for each item, the engineer marks items Real or Noise, and the marks train the next model. The `/evaluate` page runs the same engine on a team's own CSV files.

5 Conclusion

The project delivers a tested, deployable, causal on-call queue. Its measured contribution is a metric detector that catches 72.6% of GAIA faults at 20 items a day with 1.0 false item, against 68.9% and 2.1 for a Prometheus rate rule. Its main findings are negative or limited: the log-alert classifier earns its slots only where faults write logs, the full queue ties the rate rule, and all GAIA faults are one injected shape. The next step is a shadow-mode pilot with a real team.

Limitations and honest use of tools. The work is a solo project. AI coding assistants helped with code review, refactoring, tests and documentation (see `AGENTS.md`); the running system uses no generative AI. Timings come from one Windows laptop.

A Walk-forward results

Test week (2021)	Faults	With metrics	Rate rule	Detector	Full queue
8–15 Jul	161	100%	77.6%	84.5%	82.0%
15–22 Jul	134	91%	73.9%	81.3%	74.6%
22–29 Jul	162	94%	72.2%	77.2%	71.0%
29 Jul–5 Aug	163	82%	67.5%	69.3%	67.5%
5–12 Aug	162	84%	69.1%	71.6%	71.0%
12–19 Aug	139	92%	75.5%	79.1%	74.1%
19–26 Aug	173	61%	51.4%	52.0%	50.9%
26–31 Aug	129	91%	78.3%	83.7%	79.1%
Mean			70.7%	74.8%	71.3%

Table 4: Walk-forward at $K = 20$. The 19–26 Aug fold has a 2.5-day metric outage.

B Classifier on other data

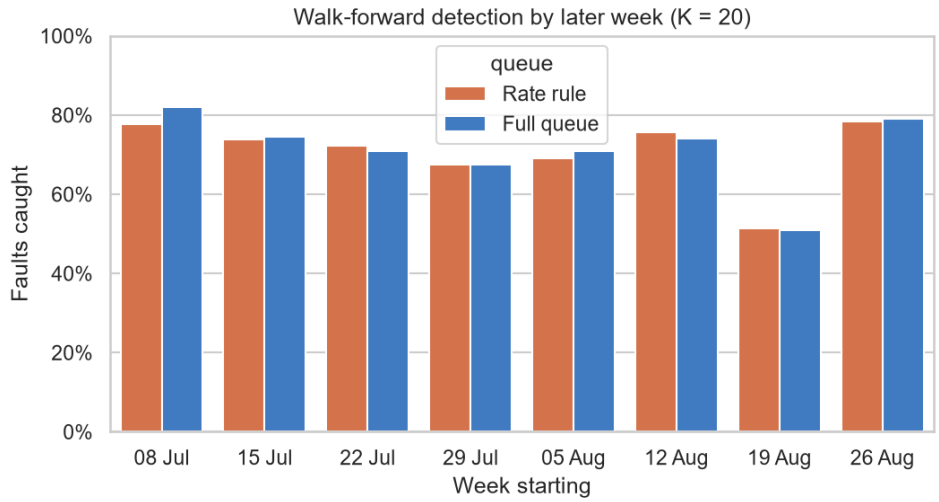
Suite	Application	Cases	No log alerts	Classifier	Gate
RE2	Online Boutique	45	44.4%	46.7%	refused
RE2	Sock Shop	45	84.4%	86.7%	refused
RE2	Train Ticket	45	60.0%	73.3%	promoted
RE3	Online Boutique	15	100%	100%	refused
RE3	Sock Shop	15	93.3%	93.3%	refused
RE3	Train Ticket	15	100%	100%	refused

Table 5: First RCAEval run: share of test faults caught at $K = 10$. The gate ran on the same cases that are scored.

Suite	Application	Cases	No log alerts	Severity	Classifier	Gate (validation)
RE2	Online Boutique	23	43.5%	47.8%	47.8%	refused
RE2	Sock Shop	23	95.7%	95.7%	95.7%	refused
RE2	Train Ticket	23	43.5%	60.9%	65.2%	refused
RE3	Online Boutique	8	100%	100%	100%	refused
RE3	Sock Shop	8	87.5%	87.5%	87.5%	refused
RE3	Train Ticket	8	100%	100%	100%	refused

Table 6: Independent gate test at $K = 10$: train, validation and test cases kept apart; the gate sees only train and validation.

C Walk-forward figure



Each bar is that week's detection rate. The spread is week-to-week variation, not a confidence interval.

Figure 3: Walk-forward detection by later week.